

Abstract class in Java

A class which is declared with the abstract keyword is known as an abstract class in **Java**. It can have abstract and non-abstract methods (method with the body).

Abstraction in Java

Abstraction is a process of hiding the implementation details and showing only functionality to the user.

Another way, it shows only essential things to the user and hides the internal details, for example, sending SMS where you type the text and send the message. You don't know the internal processing about the message delivery.

Abstraction lets you focus on what the **object** does instead of how it does it.

Ways to achieve Abstraction

There are two ways to achieve abstraction in java

1. Abstract class (0 to 100%)
2. Interface (100%)

Abstract class in Java

A class which is declared as abstract is known as an **abstract class**. It can have abstract and non-abstract methods. It needs to be extended and its method implemented. It cannot be instantiated.

Points to Remember

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have constructors and static methods also.
- It can have final methods which will force the subclass not to change the body of the method.

Example of abstract class

```
abstract class A{
```

Abstract Method in Java

A method which is declared as abstract and does not have implementation is known as an abstract method.

Example of abstract method

```
abstract void printStatus();//no method body and abstract
```

Example of Abstract class that has an abstract method

In this example, Bike is an abstract class that contains only one abstract method run. Its implementation is provided by the Honda class.

```
abstract class Bike
{
    abstract void run();
}
class Honda4 extends Bike
{
    void run()
    {
        System.out.println("running safely");
    }
    public static void main(String args[])
    {
        Bike obj = new Honda4();
        obj.run();
    }
}
running safely
```

Understanding the real scenario of Abstract class

In this example, Shape is the abstract class, and its implementation is provided by the Rectangle and Circle classes.

Mostly, we don't know about the implementation class (which is hidden to the end user), and an object of the implementation class.

In this example, if you create the instance of Rectangle class, draw() method of Rectangle class will be invoked.

abstract class Shape

```
{  
    abstract void draw();  
}
```

**//In real scenario, implementation is provided by others i.e. unknown by end user
class Rectangle extends Shape**

```
{  
    void draw()  
    {  
        System.out.println("drawing rectangle");  
    }  
}
```

class Circle1 extends Shape

```
{  
    void draw()  
    {  
        System.out.println("drawing circle");  
    }  
}
```

//In real scenario, method is called by programmer or user

class TestAbstraction1

```
{  
    public static void main(String args[])  
    {  
        Shape s=new Circle1();  
        s.draw();  
    }  
}
```

Another example of Abstract class in java

abstract class Bank

```
{  
    abstract int getRateOfInterest();  
}
```

class SBI extends Bank

```
{
```

```

        int getRateOfInterest(){return 7;}
    }
    class PNB extends Bank
    {
        int getRateOfInterest(){return 8;}
    }

    class TestBank
    {
        public static void main(String args[])
        {
            Bank b;
            b=new SBI();
            System.out.println("Rate of Interest is: "+b.getRateOfInterest()+" %");
            b=new PNB();
            System.out.println("Rate of Interest is: "+b.getRateOfInterest()+" %");
        }
    }

```

```

Rate of Interest is: 7 %
Rate of Interest is: 8 %

```

Abstract class having constructor, data member and methods

An abstract class can have a data member, abstract method, method body (non-abstract method), constructor, and even main() method.

//Example of an abstract class that has abstract and non-abstract methods

abstract class Bike

```
{
    Bike()
    {
        System.out.println("bike is created");
    }
    abstract void run();
    void changeGear()
    {
        System.out.println("gear changed");
    }
}
```

//Creating a Child class which inherits Abstract class

class Honda extends Bike

```
{
    void run()
    {
        System.out.println("running safely..");
    }
}
```

//Creating a Test class which calls abstract and non-abstract methods

class TestAbstraction2

```
{
    public static void main(String args[])
    {
        Bike obj = new Honda();
        obj.run();
        obj.changeGear();
    }
}
```

```
bike is created
    running safely..
    gear changed
```

Rule: If there is an abstract method in a class, that class must be abstract.

```
class Bike12
```

```
{
```

```
  abstract void run();
```

```
}
```

```
compile time error
```

Rule: If you are extending an abstract class that has an abstract method, you must either provide the implementation of the method or make this class abstract.